Phase 5 | Technology Study Roadmap

A sequenced curriculum tied to the implementation.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

Study sequence overview

Use 6 weeks at 45-60 minutes/day. Each week ends with a small code explanation or exercise. Essential topics come first; advanced topics are optional.

Week 1: Python, HTTP, and FastAPI (essential)

Objective: trace request input to response. Prerequisites: functions, dicts, exceptions, imports.

Learn: module imports, route decorators, path/query/body/form inputs, status codes, JSON vs HTML responses, ASGI, Uvicorn, sync/async, threadpool.

MetroInsights link: `run.py`, `app/__init__.py`, `app/routes.py`.

Exercise: describe the sequence for GET `/api/predict?station=Rajiv%20Chowk&date=...&time=09:00`; add an input validation proposal. Explain why `await run_in_threadpool(...)` is used.

Mastery: distinguish framework/server; state what async does and does not do; identify response shape and invalid-input path.

Week 2: Data manipulation and feature engineering (essential)

Objective: understand the 23 input columns. Prerequisite: Python lists/dicts and basic algebra.

Learn: DataFrames, joins, groupby, missing values, dtype conversion; vectorization; cyclical encoding; categorical IDs; calendar features; historical aggregates.

Exercise: hand-calculate hour sine/cosine for 06:00 vs 23:00; explain why both sine and cosine are needed. Trace station metadata merge and identify one data-integrity failure.

Mastery: describe each feature family, explain how runtime reconstructs same schema, spot a feature leakage path.

Week 3: ML and evaluation (essential)

Objective: defend model choice and results. Prerequisites: mean, residual, variance.

Learn: regression, decision-tree splits, boosting, regularization, MAE/RMSE/R^2/MAPE, baselines, chronological validation, leakage, distribution shift, synthetic data.

Exercise: calculate MAE for errors [2,-4,0]; compare RMSE; explain why R^2 0.9946 is not “99.46% accuracy”. Inspect errors by hour/station and propose a real-data validation split.

Mastery: accurately state target, split, sample size, metrics, baseline, and validity limitation.

Week 4: Transit capacity and planning logic (essential)

Objective: distinguish demand, capacity, clearance, headway, and occupancy.

Learn: trains/hour = 60/headway; nominal passenger capacity; through load; residual boarding clearance; utilization = estimated boardings/offered clearance; fleet and signaling constraints; peak rules.

Exercise: with clearance 800 passengers/train and 10 trains/hour, compute capacity 8,000/hour; with demand 2,000/hour utilization is 25%. Explain why this is not occupancy. Change assumed through load and explain effect on denominator.

Mastery: reproduce service recommendation decision flow; enumerate configurable assumptions; never infer actual passenger loads from the badge.

Week 5: Graphs, mapping, and algorithms (essential)

Objective: explain how a label becomes a route and how time is estimated.

Learn: canonical IDs, aliases, graph nodes, adjacency lists, weighted edges, Dijkstra, heap, complexity, Haversine, data validation.

Exercise: sketch Rajiv Chowk as Yellow and Blue nodes, add a 5-minute interchange edge; calculate route total as sum of segment and interchange edges. Construct an alias mismatch and identify skipped edges.

Mastery: explain why station-line nodes are used, Dijkstra preconditions, fallback weights, and source exclusion.

Week 6: Deployment, testing, and limitations (essential)

Objective: explain runtime dependencies and failure modes.

Learn: process startup/imports, environment port, dependency lock/pinning, joblib artifact compatibility, static assets, logs, health checks, deterministic tests.

Exercise: from a clean environment install requirements, run `uvicorn run:app --reload`, call `/api/model_metrics`, `/api/stations`, valid and invalid `/api/predict`, and a closed time. Record exact outputs. (Do this after dependencies are installable.)

Mastery: identify required files; describe eager load risk; distinguish local smoke test from deployment proof.

Advanced modules (optional)

- Pydantic typed contracts and OpenAPI schemas; contract/version management.
- SQL/PostgreSQL, data migrations, provenance and observation storage.
- Model calibration, interval prediction, residual diagnostics, drift monitoring.
- Profiling Pandas startup/memory; process workers and model replication.
- Async/background jobs only for demonstrated long-running work.
- Graph validity: GTFS ingestion, directed time-dependent edges, disruptions, station-specific transfer times.

Weekly interview checkpoint

At each week end, answer: what component did I explain; what file proves it; what assumption weakens validity; how would I verify improvement? If unable to cite code/data, mark the claim as unverified rather than memorizing it.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.